

Prueba técnica

Analista de Datos y Analítica

Respuestas escritas de las Secciones 1, 2, 5 y 6

Campo	Detalle
Candidato	Jose Alejandro Llano Cortes
Fecha de entrega	27 de septiembre de 2026
Material complementario	He decidido realizar los siguientes materiales complementarios: • BI en Power BI Desktop. • Aplicación web como material adicional a Power BI, desplegada en un servicio gratuito de Python para que pueda ser consultada por cualquier persona. • Agente conversacional realizado en n8n e integrado en la página web anteriormente mencionada.
Datos	100 % sintéticos, generados con código en Python.

Sección 1 — Power BI avanzado: modelo semántico, storytelling y criterio

Todo lo que se responde aquí está aplicado en el Power BI entregado (seccion_1_powerbi/Siniestralidad_Clientes.pbip).

1.1 Modelo semántico como capa compartida

1.1.1 ¿Qué aspectos revisaría en un modelo semántico existente para evaluar si está bien construido? Liste al menos 6 criterios con justificación.

#	Qué se revisa	Por qué
1	Estructura: tablas de hechos y de dimensiones separadas (modelo estrella)	Es la fase inicial de un proyecto: debemos revisar las fuentes de información disponibles, identificar cuáles son las tablas de hechos y de dimensiones y, posterior a ello, revisar la calidad de estas y emitir una apreciación al negocio.
2	Relaciones: sin relaciones ambiguas, sin relaciones de muchos a muchos sin justificar, ni llaves huérfanas	El modelo relacional es la base de nuestro visualizador. Debemos verificar que todas las relaciones estén correctas, ya que de lo contrario puede ocurrir que hagamos filtros o creemos medidas y estén incorrectas, pues el modelo relacional no logra capturar los datos de manera adecuada.
3	Calendario: una sola tabla de fechas, completa	Es importante para el mantenimiento del BI: de esto depende que nuestro BI pueda seguir funcionando con normalidad en fechas futuras, además de ser clave para el relacionamiento temporal.
4	Medidas: claridad, orden y simplicidad	Al desarrollar una medida la conocemos en principio, pero en revisiones futuras podemos olvidarla; por ello, las medidas deben tener nombres claros, ser lo menos complejas posible y tener documentación de cuál es su objetivo.
5	Rendimiento: tamaño, columnas que nadie usa, actualización incremental	Antes de desplegar un visualizador debemos pensar en la interacción de los usuarios: un BI con información desde el 2020 puede no ser útil y generar mucha latencia al utilizarlo. Lo mismo pasa cuando tenemos medidas sin utilizar o cuando recargamos todo el BI completo y no lo hacemos de manera incremental.
6	Seguridad: roles probados con usuarios reales	El visualizador debe mostrar la información a quien corresponde; la fuga de información a roles no autorizados es algo que no debe ocurrir.
7	Mantenimiento: versiones en Git, un dueño y pruebas contra la fuente	Es importante saber por qué se realizaron cambios en el pasado y poder volver a ellos fácilmente, además de tener una historia de construcción del BI.

1.1.2 ¿Qué diferencia hay entre un modelo semántico en Power BI Desktop, en Power BI Service y en Microsoft Fabric? ¿Cuál usaría como capa compartida y por qué?

Desktop: es donde se construye; el modelo semántico vive en un archivo en el computador y puede llegar a integrarse con Fabric.

Service: es donde se publica; se actualiza, se comparte el modelo semántico y se administran los permisos.

Fabric: también se tiene el modelo semántico, pero el consumo de información se puede hacer directamente desde el Lakehouse o el Warehouse.

Usaría Fabric. El modelo lee directo las tablas del Lakehouse, se guarda en Git y se pasa de pruebas a producción sin rehacerlo. Los reportes solo se conectan a él, así cada indicador y la seguridad se definen en un solo lugar.

1.1.3 Cuando las tablas migren de la fuente heredada al Lakehouse, ¿qué impacto tendría sobre el modelo semántico y cómo lo manejaría para que los tableros no se rompan?

Inicialmente debemos revisar que las fuentes subidas al Lakehouse conserven la estructura y nombres idénticos a las fuentes heredadas. Esto es vital porque nos ahorrará mucho trabajo y nos dará una confiabilidad alta de que ningún visualizador se rompa, pues la afectación al modelo sería casi nula: lo único que tendríamos que hacer sería cambiar el método de conexión.

Para asegurarnos de que este proceso sea correcto, debemos documentar las fuentes heredadas, realizar los cambios en el Lakehouse con base en dichas fuentes y revisarlos en un entorno de desarrollo; luego, cuando esté verificado el buen funcionamiento, se promueve al entorno de producción.

1.1.4 ¿Cómo garantizaría que las medidas DAX del modelo son consistentes y que dos áreas no calculan el mismo indicador de forma diferente?

Centralizaría los indicadores corporativos en un modelo semántico compartido y establecería una definición única para cada KPI. Las medidas DAX oficiales vivirían en ese modelo y los reportes consumirían esas medidas en lugar de recrearlas localmente. Además, documentaría para cada indicador su fórmula, filtros, granularidad, fuente y responsable, y establecería un proceso de revisión para cambios.

Sin embargo, eso funciona solo en un escenario perfecto, pues en algunas ocasiones, por ejemplo, las ventas para un área involucran unos conceptos, pero para otras áreas involucran otros conceptos, lo cual hace que los datos cambien entre áreas. Entonces, la parte clave es que las áreas no puedan publicar un BI sin una revisión previa del modelo semántico compartido oficial y, en caso de que no se ajusten a esos criterios, solicitar agregar una nueva forma de revisar ese indicador.

1.2 Dataflows y fuente única para la tasa de incidencia

El equipo tiene 7 reportes que calculan la tasa de incidencia de formas diferentes. Nadie sabe cuál es el correcto y los números no coinciden en las reuniones de gerencia.

1.2.1 ¿Cómo usaría los Dataflows de Power BI para resolver este problema de raíz? ¿Dónde viviría la definición oficial de tasa de incidencia?

Inicialmente se debe tener una reunión con el dueño del proceso y acordar cuál debe ser la limpieza y transformación de datos que nos asegure tener únicamente la información verídica que se debe analizar. Cuando tengamos esto, realizamos el Dataflow para tener una única fuente de información.

Posterior a ello, se debe socializar esta transformación de datos, incluirla en el modelo semántico oficial de la compañía y promover el uso exclusivo de esta fórmula.

1.2.2 ¿Cuándo preferiría que esa lógica viviera en el Lakehouse en lugar del Dataflow? ¿Qué factores consideraría?

Preferiría que estuviera en Dataflows si la cantidad de registros no es grande y se usará exclusivamente para Power BI. Si la información es de utilidad para otros procesos y tiene un proceso de transformación grande, debería ir en el Lakehouse por eficiencia, pues es el lugar donde reposa la información de la compañía de manera centralizada.

1.2.3 ¿Cómo le explicaría a un gerente, sin usar términos técnicos, por qué ahora todos los reportes darán el mismo número?

Anteriormente no había una política de gobierno de datos estructurada, lo que permitía que cada área transformara los datos según su propia lógica de negocio. Ahora se implementó una arquitectura que centraliza las fuentes de información, realiza el proceso de transformación de los datos y los pone a disposición en un solo lugar, facilitándoles a los procesos el uso de indicadores estandarizados. Así nos aseguramos de que todas las personas utilicemos las mismas fuentes de información y, por ende, los resultados sean los mismos.

1.3 Storytelling y tableros que se explican solos

1.3.1 ¿Qué principios seguiría para construir un tablero que cuente una historia coherente de principio a fin? Mencione al menos 5 principios concretos.

- **Cada página responde una pregunta:** ¿cómo vamos?, ¿dónde está el problema?, ¿qué debo priorizar?
- **De lo general a lo particular:** primero debemos saber qué indicadores presentan bajo rendimiento; posterior a ello, debemos buscar cuáles son los componentes que están afectando en mayor medida ese indicador.
- **Indicadores compuestos:** los indicadores deben estar acompañados de variaciones y cumplimientos; tener el indicador no me responde si estoy mejorando, empeorando o si no estoy cumpliendo con lo presupuestado.
- **Diseño intuitivo:** más gráficas, más colores y variaciones, menos tablas.
- **Simplicidad como grado de sofisticación:** entre más simple, claro y conciso sea un visualizador, mayor será su capacidad de generar valor para el negocio. El objetivo de los BI debe ser facilitar la comprensión de la información y apoyar la toma oportuna de decisiones. Un tablero excesivamente complejo, sobrecargado de elementos o difícil de interpretar puede reducir su usabilidad y, en consecuencia, limitar su aporte al negocio.

1.3.2 ¿Cómo decidiría qué métricas van en la primera página vs en páginas de detalle?

Primera página: aquí deben estar los indicadores principales de la compañía, es decir, aquellos que realmente nos muestran si el negocio está funcionando bien o si hay alguna situación que pueda generar un impacto o detrimento para la empresa. La idea es que esta página permita entender rápidamente qué está pasando y tomar decisiones.

- **Páginas de detalle:** en estas páginas podemos profundizar en los indicadores de la primera página y entender por qué están pasando las cosas. Son más enfocadas en el detalle y en la operación interna del proceso, por lo que no necesariamente deben tener el mismo enfoque gerencial de la primera página.

1.3.3 Un gerente le dice que el tablero 'se ve bonito, pero no le sirve para tomar decisiones'. ¿Qué preguntas le haría y qué cambiaría?

Le haría las siguientes preguntas:

¿Cuál es la problemática actual del proceso?

¿Qué decisión debería ayudar a tomar?

¿A quiénes y a cuál proceso buscamos impactar?

¿La solución es un tablero o es otra herramienta digital?

1.4 Acceso por perfil de usuario (RLS)

1.4.1 ¿Implementaría seguridad estática o dinámica? Justifique y describa la tabla que necesitaría en el modelo.

Implementaría seguridad dinámica. Con 60 coordinadores y cambios frecuentes, la estática obligaría a mantener 60 roles y republicar con cada cambio. Con la dinámica hay un solo rol: el modelo sabe quién está consultando por su correo y le muestra solo sus clientes, según una tabla de asignaciones.

Columna de la tabla de asignaciones	Qué guarda
correo	Correo del coordinador
id_cliente	Cliente asignado (una fila por coordinador y cliente)
vigente_desde / vigente_hasta	Desde y hasta cuándo aplica, para saber quién tenía qué
fuelle / fecha_actualizacion	De dónde sale (el CRM, no un Excel a mano) y cuándo se cargó

El filtro del rol queda así:

```
[id_cliente] IN CALCULATETABLE ( VALUES ( 'Seguridad Asignacion'[id_cliente] ),  
                                     'Seguridad Asignacion'[Correo] = LOWER ( USERPRINCIPALNAME ( ) ) )
```

1.4.2 ¿Cómo verificaría que la seguridad funciona correctamente antes de publicar?

Entrando con el perfil de varios coordinadores reales (Power BI lo permite) y revisando que cada uno vea justo sus clientes.

1.4.3 ¿Qué pasa cuando la asignación de un cliente cambia? ¿Qué debe actualizarse?

Solo se cambia la fila en la tabla de asignaciones. Cuando el modelo se actualiza, el nuevo coordinador ve el cliente y el anterior deja de verlo, sin tocar roles ni republicar.

1.5 Medidas DAX

Medida A: Casos YTD = TOTALYTD(COUNT(casos[id_caso]), calendario[fecha])

Medida B: % participación = DIVIDE([Total casos], CALCULATE([Total casos], ALL(clientes)), 0)

Medida C: Var vs mes anterior =

[Casos mes actual] - CALCULATE([Casos mes actual], PREVIOUSMONTH(calendario[fecha]))

1.5.1 Explique en términos de negocio qué calcula cada medida y cuándo la usaría. Incluya el comportamiento cuando no hay datos:

Medida A — Casos YTD:

Qué calcula: los casos acumulados en lo que va del año.

Cuándo usarla: para comparar contra una meta anual o contra el mismo punto del año pasado.

Sin datos: queda vacía, no en 0.

Medida B — % participación:

Qué calcula: qué parte del total de casos tiene cada cliente, sector o clase.

Cuándo usarla: para ver dónde se concentran los casos.

Sin datos: muestra 0 % en los clientes sin casos; prefiero dejarlos vacíos para que no aparezcan.

Medida C — Variación vs mes anterior:

Qué calcula: cuántos casos más o menos hubo que en el mes anterior.

Cuándo usarla: en el seguimiento de cada mes.

1.5.2 ¿Por qué es necesario ALL(clientes) en la Medida B? ¿Qué pasaría si no estuviera?

¿Por qué ALL(clientes)?

- Porque el denominador tiene que ser el total de todos los clientes, no solo el de la fila.
- Sin ALL, el numerador y el denominador dan lo mismo y todas las filas salen en 100 %.

1.5.3 ¿Qué devuelve la Medida C cuando no hay datos del mes anterior? ¿Cómo lo manejaría?

Sin mes anterior, resta contra vacío y muestra el mes completo como si fuera crecimiento: enero de 2024, el primer mes, sale con +373, y eso es falso.

Cómo lo manejo: si no hay mes anterior, la dejo vacía con el aviso 'Sin dato del mes anterior'; si hubo cero casos, muestra 0; y solo la calculo cuando se mira un mes:

```
C corregida =  
VAR_act = [Total casos] + 0  
VAR_hayAnterior = NOT ISEMPTY ( PREVIOUSMONTH ( Calendario[Fecha] ) )  
VAR_ant = CALCULATE ( [Total casos], PREVIOUSMONTH ( Calendario[Fecha] ) ) + 0  
RETURN IF ( _hayAnterior && HASONEVALUE ( Calendario[Inicio mes] ), _act - _ant )
```

Sección 2 — SQL: construcción, pipelines y lectura de lógica

Todas las consultas se corrieron sobre los datos de prueba y se compararon con un cálculo hecho aparte en Python: 19 de 19 resultados coinciden.

2.1 Lectura y documentación de lógica existente

Le entregan la siguiente consulta de un colega que ya no está en el equipo. Debe entenderla, identificar problemas y documentarla para publicarla en la biblioteca compartida de consultas.

2.1.1 Explique en dos párrafos, sin términos técnicos, qué está calculando esta consulta y para qué sirve.

La consulta revisa los clientes activos que tuvieron algún incidente en el último año a través de estos indicadores: cantidad de incidentes, cantidad de incidentes graves, costo total y trabajadores promedio. Con eso saca el porcentaje de graves y cuánto cuestan los incidentes por trabajador.

Luego ordena a los clientes del que más cuesta al que menos, comparando solo con los de su mismo nivel de riesgo.

Sirve para saber qué clientes nos están costando más y en cuáles enfocar la gestión.

2.1.2 Identifique al menos un problema técnico que afecta la precisión del resultado. Explíquelo y proponga la corrección.

Problema: la consulta está uniando datos con diferente nivel de detalle, ya que agrupamos a los clientes para tener un solo registro de los últimos 12 meses, pero la facturación no tiene ninguna agregación; entonces, los datos se multiplicarán tantas veces como filas tenga la tabla de facturación. Además, la tabla de facturación solo tiene en cuenta los últimos 6 meses.

Solución: resumir cada tabla por cliente por separado y después juntarlas.

Además:

- Marca como grave lo que tiene más de 15 días de ausencia, en vez de usar el campo tipo (190 casos quedan mal).
- No quita los casos anulados.

Consulta corregida (seccion_2/2_1_consulta_corregida.sql):

```
-- =====
-- Biblioteca compartida de consultas
-- Nombre      : siniestralidad_12m_por_cliente_activo
-- Versión     : 2.0 (corrige v1.0 heredada)
-- Propósito   : Resumir, por cliente activo, los incidentes de los últimos 12 meses
-- Tablas      : clientes, casos, facturacion
-- Parámetros  : fecha_corte (por defecto: día anterior a la ejecución)
-- =====

WITH params AS (
    SELECT CAST(CURRENT_DATE - INTERVAL 1 DAY AS DATE) AS fecha_corte
),
ventana AS (
    SELECT fecha_corte,
           CAST(fecha_corte - INTERVAL 12 MONTH AS DATE) AS desde_casos,
           CAST(DATE_TRUNC('month', fecha_corte) - INTERVAL 11 MONTH AS DATE) AS desde_periodo
    FROM params
),
casos_12m AS (
    SELECT ca.id_cliente,
           COUNT(*) AS total_casos,
           SUM(CASE WHEN ca.tipo = 'grave' THEN 1 ELSE 0 END) AS casos_graves,
           SUM(COALESCE(ca.costo, 0)) AS costo_total,
           -- 1 fila por cliente
    FROM clientes ca
    JOIN casos c ON ca.id_cliente = c.id_cliente
    WHERE c.fecha_corte <= fecha_corte
    GROUP BY ca.id_cliente
)
```

```

SUM(CASE WHEN ca.costo IS NULL THEN 1 ELSE 0 END) AS casos_sin_costo
FROM casos ca CROSS JOIN ventana v
WHERE ca.fecha_ocurrencia > v.desde_casos
AND ca.fecha_ocurrencia <= v.fecha_corte
AND ca.estado <> 'anulado'
GROUP BY ca.id_cliente
),
trabajadores_12m AS ( -- 1 fila por cliente
SELECT f.id_cliente,
AVG(f.trabajadores_activos) AS trabajadores_prom,
COUNT(*) AS periodos_facturados
FROM facturacion f CROSS JOIN ventana v
WHERE f.periodo >= v.desde_periodo
AND f.periodo <= v.fecha_corte
GROUP BY f.id_cliente
),
base AS (
SELECT c.id_cliente, c.nombre, c.sector, c.clase_riesgo,
ca.total_casos, ca.casos_graves, ca.costo_total, ca.casos_sin_costo,
t.trabajadores_prom, t.periodos_facturados
FROM clientes c
JOIN casos_12m ca ON ca.id_cliente = c.id_cliente
LEFT JOIN trabajadores_12m t ON t.id_cliente = c.id_cliente
WHERE c.estado = 'activo'
)
SELECT id_cliente, nombre, sector, clase_riesgo,
total_casos, casos_graves, costo_total,
ROUND(trabajadores_prom, 1) AS trabajadores_prom,
ROUND(casos_graves * 100.0 / NULLIF(total_casos, 0), 1) AS pct_graves,
ROUND(costo_total / NULLIF(trabajadores_prom, 0), 0) AS costo_x_trab,
DENSE_RANK() OVER (PARTITION BY clase_riesgo
ORDER BY costo_total DESC) AS ranking_clase,
periodos_facturados, -- < 12 => costo_x_trab menos confiable
casos_sin_costo -- > 0 => costo_total subestimado
FROM base
ORDER BY costo_total DESC, id_cliente;

```

2.1.3 Escriba la documentación de esta consulta en el formato que usaría para publicarla en la biblioteca compartida: nombre, propósito, tablas fuente, campos calculados y notas de uso.

Campo	Contenido
Nombre	siniestralidad_12m_por_cliente_activo (versión 2, reemplaza la heredada)
Propósito	Ver, por cliente activo, los incidentes del último año, su costo y su costo por trabajador, con un puesto dentro de su nivel de riesgo.
Tablas fuente	clientes, casos y facturacion.
Parámetro	fecha_corte (por defecto, ayer). Se miran los 12 meses anteriores.
Reglas	Solo clientes activos y casos no anulados. Grave = tipo 'grave'.
Campos calculados	total_casos, casos_graves, pct_graves, costo_total, trabajadores_prom, costo_x_trab (costo por trabajador, sirve para comparar clientes de distinto tamaño) y ranking_clase (1 = el más costoso de su nivel).
Notas de uso	No sumar costo_x_trab entre clientes. Los clientes nuevos tienen menos historia. Con la misma fecha de corte siempre da lo mismo.

2.2 Construcción de consulta avanzada

El equipo necesita cada semana los 15 clientes que más atención prioritaria requieren para planear visitas. Construya la consulta SQL usando las tablas del contexto.

2.2.1 Criterio de priorización y justificación de cada criterio

Una visita vale la pena donde el riesgo es alto y hay algo que hacer. Si ordenara solo por número de casos, siempre saldrían los clientes más grandes. Por eso uso un puntaje con siete criterios:

Criterio	Peso	Por qué
Frecuencia: casos por cada 100 trabajadores (90 días)	30 %	Es lo que mejor muestra el riesgo actual
Severidad: días de ausencia por trabajador (90 días)	20 %	No es lo mismo muchos casos leves que pocos graves
Tendencia: últimos 90 días contra los 90 anteriores	15 %	Hay que llegar antes de que el cliente se vuelva crítico
Costo por trabajador (12 meses)	10 %	El impacto económico, sin que pese el tamaño
Casos graves abiertos	10 %	Hay algo concreto que revisar en la visita
Días sin actividades de prevención	10 %	Donde no hemos ido es donde más sirve ir
Clase de riesgo	5 %	Pesa poco porque los demás criterios ya la reflejan

2.2.2 Consulta SQL de priorización

```
-- =====
-- Nombre      : top15_clientes_atencion_prioritaria
-- Uso         : Ejecución semanal (lunes) para planear visitas de la semana.
-- Criterio: un cliente necesita atención prioritaria cuando se accidenta MÁS de lo
-- esperado, con incidentes MÁS serios, está EMPEORANDO, le cuesta más por trabajador,
-- tiene casos graves aún ABIERTOS y lleva tiempo SIN prevención. Cada criterio se
-- convierte en un percentil (0 a 1) entre los clientes activos para que sean
-- comparables sin importar su unidad, y se ponderan así:
--
-- Componente                                Peso  Ventana
-- 1. Frecuencia: tasa de casos ajustada      30 %  últimos 90 días
-- 2. Severidad: días perdidos por trabajador  20 %  últimos 90 días
-- 3. Tendencia: casos 90 d vs 90 d anteriores 15 %  180 días
-- 4. Costo por trabajador                    10 %  últimos 12 meses
-- 5. Casos graves abiertos                   10 %  a la fecha de corte
-- 6. Brecha de prevención (días sin actividad) 10 %  a la fecha de corte
-- 7. Clase de riesgo (1-5)                   5 %  estructural
--
-- Ajuste por tamaño (componente 1): la tasa se "suaviza" hacia el promedio de su
-- clase de riesgo con una exposición de referencia de 300 trabajadores-mes. Así un
-- cliente de 15 trabajadores con 1 caso no sube al primer lugar solo por azar,
-- mientras que un cliente grande con tasa alta sí lo hace.
-- =====
WITH params AS (
    SELECT CAST(CURRENT_DATE - INTERVAL 1 DAY AS DATE) AS fecha_corte, -- ayer: fecha de corte
           300.0 AS exposicion_ref -- trabajadores-mes
),
activos AS (
    SELECT id_cliente, nombre, sector, clase_riesgo
    FROM clientes WHERE estado = 'activo'
),
casos_validos AS (
    SELECT ca.*
    FROM casos ca CROSS JOIN params p
    WHERE ca.estado <> 'anulado'
      AND ca.fecha_ocurrencia <= p.fecha_corte
      AND ca.fecha_ocurrencia > CAST(p.fecha_corte - INTERVAL 12 MONTH AS DATE)
),
casos_agg AS (
    SELECT cv.id_cliente,
           SUM(CASE WHEN cv.fecha_ocurrencia > p.fecha_corte - 90 THEN 1 ELSE 0 END) AS casos_90d,
           SUM(CASE WHEN cv.fecha_ocurrencia <= p.fecha_corte - 90
                        AND cv.fecha_ocurrencia > p.fecha_corte - 180 THEN 1 ELSE 0 END) AS casos_prev_90d,
           SUM(CASE WHEN cv.fecha_ocurrencia > p.fecha_corte - 90 THEN cv.dias_ausencia ELSE 0 END) AS dias_90d,
           SUM(CASE WHEN cv.fecha_ocurrencia > p.fecha_corte - 90 AND cv.tipo = 'grave' THEN 1 ELSE 0 END) AS graves_90d,
           SUM(CASE WHEN cv.tipo = 'grave' AND cv.estado = 'abierto' THEN 1 ELSE 0 END) AS graves_abiertos,
           SUM(COALESCE(cv.costo, 0)) AS costo_12m
    FROM casos_validos cv CROSS JOIN params p
    GROUP BY cv.id_cliente
),
trab AS ( -- trabajadores promedio: últimos 3 periodos (90 d) y últimos 12 periodos (costo)
    SELECT f.id_cliente,
           AVG(CASE WHEN f.periodo >= CAST(DATE_TRUNC('month', p.fecha_corte) - INTERVAL 2 MONTH AS DATE)
```

```

        THEN f.trabajadores_activos END) AS trab_3m,
        AVG(f.trabajadores_activos) AS trab_12m
    FROM facturacion f CROSS JOIN params p
    WHERE f.periodo >= CAST(DATE_TRUNC('month', p.fecha_corte) - INTERVAL 11 MONTH AS DATE)
        AND f.periodo <= p.fecha_corte
    GROUP BY f.id_cliente
),
prev AS (
    SELECT pr.id_cliente, MAX(pr.fecha) AS ultima_prevision
    FROM prevision pr CROSS JOIN params p
    WHERE pr.fecha <= p.fecha_corte
    GROUP BY pr.id_cliente
),
base AS (
    SELECT a.*,
        COALESCE(ca.casos_90d, 0) AS casos_90d,
        COALESCE(ca.casos_prev_90d, 0) AS casos_prev_90d,
        COALESCE(ca.dias_90d, 0) AS dias_90d,
        COALESCE(ca.graves_90d, 0) AS graves_90d,
        COALESCE(ca.graves_abiertos, 0) AS graves_abiertos,
        COALESCE(ca.costo_12m, 0) AS costo_12m,
        t.trab_3m, t.trab_12m,
        t.trab_3m * 3 AS exposicion_3m, -- trabajadores-mes
        pv.ultima_prevision,
        -- sin prevención registrada => se trata como 365 días (máxima brecha)
        COALESCE(CAST(p.fecha_corte - pv.ultima_prevision AS INTEGER), 365) AS dias_sin_prevision
    FROM activos a
    CROSS JOIN params p
    JOIN trab t ON t.id_cliente = a.id_cliente -- exige facturación reciente
    LEFT JOIN casos_agg ca ON ca.id_cliente = a.id_cliente
    LEFT JOIN prev pv ON pv.id_cliente = a.id_cliente
    WHERE t.trab_3m > 0
),
tasa_clase AS ( -- tasa mensual por cada 100 trabajadores, promedio de la clase
    SELECT clase_riesgo, SUM(casos_90d) * 100.0 / SUM(exposicion_3m) AS tasa_clase
    FROM base GROUP BY clase_riesgo
),
metricas AS (
    SELECT b.*, tc.tasa_clase,
        b.casos_90d * 100.0 / b.exposicion_3m AS tasa_90d,
        (b.casos_90d + p.exposicion_ref * tc.tasa_clase / 100.0) * 100.0 / (b.exposicion_3m + p.exposicion_ref) AS tasa_ajustada,
        b.dias_90d * 1.0 / b.trab_3m AS dias_perdidos_x_trab,
        (b.casos_90d + 1.0) / (b.casos_prev_90d + 1.0) AS indice_tendencia,
        b.costo_12m / b.trab_12m AS costo_x_trab_12m
    FROM base b JOIN tasa_clase tc USING (clase_riesgo) CROSS JOIN params p
),
percentiles AS (
    SELECT m.*,
        PERCENT_RANK() OVER (ORDER BY tasa_ajustada) AS p_frecuencia,
        PERCENT_RANK() OVER (ORDER BY dias_perdidos_x_trab) AS p_severidad,
        PERCENT_RANK() OVER (ORDER BY indice_tendencia) AS p_tendencia,
        PERCENT_RANK() OVER (ORDER BY costo_x_trab_12m) AS p_costo,
        PERCENT_RANK() OVER (ORDER BY graves_abiertos) AS p_abiertos,
        PERCENT_RANK() OVER (ORDER BY dias_sin_prevision) AS p_prevision,
        (clase_riesgo - 1) / 4.0 AS p_clase
    FROM metricas m
),
puntaje AS (
    SELECT p.*,
        0.30 * p_frecuencia AS c_frecuencia, 0.20 * p_severidad AS c_severidad,
        0.15 * p_tendencia AS c_tendencia, 0.10 * p_costo AS c_costo,
        0.10 * p_abiertos AS c_abiertos, 0.10 * p_prevision AS c_prevision,
        0.05 * p_clase AS c_clase
    FROM percentiles p
),
final AS (
    SELECT pu.*,
        ROUND(100 * (c_frecuencia + c_severidad + c_tendencia + c_costo
            + c_abiertos + c_prevision + c_clase), 1) AS puntaje_prioridad
    FROM puntaje pu
)
SELECT ROW_NUMBER() OVER (ORDER BY puntaje_prioridad DESC, tasa_ajustada DESC, id_cliente) AS prioridad,
    id_cliente, nombre, sector, clase_riesgo,
    puntaje_prioridad,
    ROUND(tasa_90d, 2) AS tasa_mensual_90d,
    ROUND(tasa_ajustada, 2) AS tasa_ajustada,
    casos_90d, casos_prev_90d, graves_90d, graves_abiertos,
    ROUND(dias_perdidos_x_trab, 2) AS dias_perdidos_x_trab,
    ROUND(costo_x_trab_12m, 0) AS costo_x_trab_12m,
    dias_sin_prevision,
    -- Motivo principal: el criterio en el que el cliente está más alto frente a los

```

```

-- demás (mayor percentil). Le dice al coordinador QUÉ revisar en la visita.
CASE GREATEST(p_frecuencia, p_severidad, p_tendencia, p_costo, p_abiertos, p_preencion)
  WHEN p_frecuencia THEN 'Alta frecuencia de casos'
  WHEN p_severidad THEN 'Casos severos (días perdidos)'
  WHEN p_tendencia THEN 'Casos en aumento'
  WHEN p_costo THEN 'Alto costo por trabajador'
  WHEN p_abiertos THEN 'Casos graves abiertos'
  ELSE 'Sin prevención reciente' END AS motivo_principal
FROM final
ORDER BY prioridad
LIMIT 15;

```

Resultado con los datos de prueba:

#	Cliente	Clase	Puntaje	Tasa aj.	Casos 90 d	Graves abiertos	Días sin prev.	Motivo principal
1	Nogal Minería S.A.S.	4	95,9	5,07	22	7	527	Alta frecuencia de casos
2	Orinoco Construcción S.A.	4	90,4	4,34	32	9	26	Alta frecuencia de casos
3	Horizonte Construcción Group S.A.S.	4	88,8	3,46	69	18	74	Casos graves abiertos
4	Farallones Construcción Ltda.	5	88,0	3,56	14	3	585	Alto costo por trabajador
5	Pacífico Agroindustria Ltda.	4	85,4	2,54	10	5	24	Alto costo por trabajador
6	Nogal Manufactura S.A.S.	4	84,7	3,28	27	5	11	Alto costo por trabajador
7	Zenú Manufactura S.A.	4	84,6	3,42	13	4	2	Alta frecuencia de casos
8	Tayrona Minería & Cía.	4	82,9	3,25	12	2	6	Casos severos (días perdidos)
9	Orinoco Agroindustria S.A.	3	82,6	2,97	62	7	153	Casos graves abiertos
10	Palmira Construcción Group S.A.S.	4	82,1	3,34	8	3	5	Alta frecuencia de casos
11	Farallones Manufactura Ltda.	4	81,1	2,56	9	2	21	Casos severos (días perdidos)
12	Tequendama Construcción Group S.A.S.	4	80,5	2,83	88	25	7	Casos graves abiertos
13	Palmira Agroindustria S.A.S.	4	79,0	2,96	11	1	26	Alta frecuencia de casos
14	Andina Transporte S.A.	4	76,6	2,26	13	3	11	Casos severos (días perdidos)
15	Nogal Agroindustria S.A.S.	4	76,5	3,10	14	3	65	Alta frecuencia de casos

2.3 Pipeline de datos

Construya una consulta SQL base para un pipeline diario que alimenta el tablero ejecutivo. Debe calcular para cada cliente activo, a corte del día anterior: total de casos del mes en curso y variación vs el mes anterior; tasa de incidencia; actividades de prevención de los últimos 30 días; y clasificación crítico, moderado o bajo.

2.3.1 Consulta SQL del pipeline diario

- La fecha de corte es un parámetro (ayer), así se puede volver a correr cualquier día.
- Además de la variación contra el mes anterior completo, la calculo contra los mismos días del mes anterior. El 26 de septiembre había 284 casos: contra los 322 de todo agosto parece una mejora del 12 %, pero contra el 1–26 de agosto (274) en realidad es un aumento del 4 %.
- Si un cliente todavía no tiene facturación del mes, uso sus trabajadores del mes anterior y lo marco.
- Clasificación: crítico si la tasa es mayor a 5, moderado de 2 a 5 y bajo si es menor a 2. Hoy: 3 críticos, 25 moderados y 78 bajos entre 106 clientes activos (igual que en el Power BI).

```

-- =====
-- Nombre      : pipeline_diario_tablero_ejecutivo (capa gold: gold.indicadores_cliente_diario)
-- Frecuencia  : diaria, 05:00, a corte del día anterior.
-- Reglas de negocio:
-- * Caso válido = estado <> 'anulado'.

```

```

-- * Mes en curso = del día 1 del mes de la fecha de corte hasta la fecha de corte.
-- * Mes anterior = mes calendario anterior completo (lo que pide el requerimiento).
-- Se agrega además el mes anterior "al mismo corte" (mismos días transcurridos),
-- que es la comparación justa mientras el mes en curso está incompleto.
-- * Tasa de incidencia = casos del mes / trabajadores activos del mes * 100.
-- Trabajadores activos = facturacion.trabajadores_activos del periodo del mes en
-- curso; si aún no se ha facturado, se usa el último periodo disponible y se marca.
-- * Prevención 30 días = actividades con fecha en (corte - 30, corte].
-- * Clasificación: 'crítico' tasa > 5 | 'moderado' 2 <= tasa <= 5 | 'bajo' tasa < 2
-- | 'sin dato' si no hay trabajadores activos (no se puede calcular la tasa).
-- =====
WITH params AS (
    SELECT CAST(CURRENT_DATE - INTERVAL 1 DAY AS DATE) AS fecha_corte -- ayer: fecha de corte
),
fechas AS (
    SELECT fecha_corte,
           CAST(DATE_TRUNC('month', fecha_corte) AS DATE) AS ini_mes,
           CAST(DATE_TRUNC('month', fecha_corte) - INTERVAL 1 MONTH AS DATE) AS ini_mes_ant,
           CAST(DATE_TRUNC('month', fecha_corte) - INTERVAL 1 DAY AS DATE) AS fin_mes_ant,
           -- mismo día del mes anterior (acotado al último día si el mes anterior es más corto)
           LEAST(CAST(fecha_corte - INTERVAL 1 MONTH AS DATE),
                CAST(DATE_TRUNC('month', fecha_corte) - INTERVAL 1 DAY AS DATE)) AS corte_mes_ant,
           EXTRACT(DAY FROM fecha_corte) AS dias_transcurridos,
           EXTRACT(DAY FROM CAST(DATE_TRUNC('month', fecha_corte) + INTERVAL 1 MONTH
                                     - INTERVAL 1 DAY AS DATE)) AS dias_mes
    FROM params
),
clientes_activos AS (
    SELECT id_cliente, nombre, sector, clase_riesgo
    FROM clientes WHERE estado = 'activo'
),
casos_mes AS (
    SELECT ca.id_cliente,
           SUM(CASE WHEN ca.fecha_ocurrencia BETWEEN f.ini_mes AND f.fecha_corte THEN 1 ELSE 0 END) AS casos_mes,
           SUM(CASE WHEN ca.fecha_ocurrencia BETWEEN f.ini_mes_ant AND f.fin_mes_ant THEN 1 ELSE 0 END) AS
casos_mes_anterior,
           SUM(CASE WHEN ca.fecha_ocurrencia BETWEEN f.ini_mes_ant AND f.corte_mes_ant THEN 1 ELSE 0 END) AS
casos_mes_ant_mismo_corte,
           SUM(CASE WHEN ca.fecha_ocurrencia BETWEEN f.ini_mes AND f.fecha_corte
                        AND ca.tipo = 'grave' THEN 1 ELSE 0 END) AS casos_graves_mes
    FROM casos ca CROSS JOIN fechas f
    WHERE ca.estado <> 'anulado'
    AND ca.fecha_ocurrencia BETWEEN f.ini_mes_ant AND f.fecha_corte
    GROUP BY ca.id_cliente
),
facturacion_rank AS ( -- periodo del mes en curso o, si falta, el último anterior
    SELECT fa.id_cliente, fa.periodo, fa.trabajadores_activos,
           ROW_NUMBER() OVER (PARTITION BY fa.id_cliente ORDER BY fa.periodo DESC) AS rn
    FROM facturacion fa CROSS JOIN fechas f
    WHERE fa.periodo <= f.ini_mes
),
trabajadores AS (
    SELECT id_cliente, periodo AS periodo_trabajadores, trabajadores_activos
    FROM facturacion_rank WHERE rn = 1
),
prevencion_30d AS (
    SELECT pr.id_cliente, COUNT(*) AS actividades_prevencion_30d,
           SUM(pr.participantes) AS participantes_prevencion_30d
    FROM prevencion pr CROSS JOIN fechas f
    WHERE pr.fecha > CAST(f.fecha_corte - INTERVAL 30 DAY AS DATE)
    AND pr.fecha <= f.fecha_corte
    GROUP BY pr.id_cliente
),
calculos AS (
    SELECT f.fecha_corte, c.id_cliente, c.nombre, c.sector, c.clase_riesgo,
           COALESCE(cm.casos_mes, 0) AS casos_mes,
           COALESCE(cm.casos_mes_anterior, 0) AS casos_mes_anterior,
           COALESCE(cm.casos_mes_ant_mismo_corte, 0) AS casos_mes_ant_mismo_corte,
           COALESCE(cm.casos_graves_mes, 0) AS casos_graves_mes,
           t.trabajadores_activos,
           t.periodo_trabajadores,
           (t.periodo_trabajadores < f.ini_mes) AS trabajadores_de_periodo_anterior,
           COALESCE(p.actividades_prevencion_30d, 0) AS actividades_prevencion_30d,
           COALESCE(p.participantes_prevencion_30d, 0) AS participantes_prevencion_30d,
           f.dias_transcurridos, f.dias_mes
    FROM clientes_activos c
    CROSS JOIN fechas f
    LEFT JOIN casos_mes cm ON cm.id_cliente = c.id_cliente
    LEFT JOIN trabajadores t ON t.id_cliente = c.id_cliente
    LEFT JOIN prevencion_30d p ON p.id_cliente = c.id_cliente
),
indicadores AS (

```

```

SELECT *,
    casos_mes - casos_mes_anterior                                AS var_casos_vs_mes_ant,
    ROUND((casos_mes - casos_mes_anterior) * 100.0               AS var_pct_vs_mes_ant,
          / NULLIF(casos_mes_anterior, 0), 1)                    AS var_casos_vs_mismo_corte,
    casos_mes - casos_mes_ant_mismo_corte                        AS tasa_incidencia
    casos_mes * 100.0 / NULLIF(trabajadores_activos, 0)
FROM calculo
)
SELECT fecha_corte, id_cliente, nombre, sector, clase_riesgo,
    casos_mes, casos_mes_anterior, var_casos_vs_mes_ant, var_pct_vs_mes_ant,
    casos_mes_ant_mismo_corte, var_casos_vs_mismo_corte,
    casos_graves_mes,
    trabajadores_activos, periodo_trabajadores, trabajadores_de_periodo_anterior,
    ROUND(tasa_incidencia, 2)                                     AS tasa_incidencia,
    -- Informativa: tasa proyectada a mes completo (no se usa para clasificar)
    ROUND(tasa_incidencia * dias_mes / dias_transcurridos, 2)    AS tasa_proyectada_mes,
    actividades_prevenccion_30d, participantes_prevenccion_30d,
    CASE WHEN tasa_incidencia IS NULL THEN 'sin dato'
        WHEN tasa_incidencia > 5 THEN 'critico'
        WHEN tasa_incidencia >= 2 THEN 'moderado'
        ELSE 'bajo' END                                           AS clasificacion,
    CURRENT_TIMESTAMP                                             AS fecha_proceso
FROM indicadores
ORDER BY tasa_incidencia DESC NULLS LAST, id_cliente;

```

2.3.2 Adicional: ¿cómo garantizaría que este pipeline detecta automáticamente cuando la calidad del dato cayó antes de que el resultado llegue al tablero?

El pipeline **no escribe directo en el tablero**: primero arma el resultado aparte, lo revisa y solo si pasa lo publica. Si algo falla, el tablero se queda con la última versión buena (con su fecha a la vista) y le llega un aviso al responsable. Lo que revisa (seccion_2/2_3_controles_calidad.sql):

Revisión	Qué mira	Si falla
Datos al día	Que el último caso cargado no tenga más de 2 días de atraso	No publica
Carga completa	Que los casos del día no caigan de golpe frente al promedio	Avisa
Datos sanos	Sin vacíos en campos clave, sin casos repetidos, sin valores negativos ni fechas futuras	No publica
Todo cuadra	Una fila por cliente activo y el total de casos igual al de la fuente	No publica
Tasas lógicas	Que ninguna tasa quede por fuera de 0 a 100	No publica

Sección 5 — Inventario y migración de reportes existentes

5.1 Análisis y plan de migración

Recibe un inventario de 10 reportes existentes. Su tarea es analizarlos y definir un plan de migración a Power BI.

5.1.1 ¿Cuáles migra primero y por qué? ¿Cuáles descartaría sin migrar? Justifique cada decisión.

Antes de migrar cualquier reporte, debemos analizar si el reporte es de utilidad para la compañía y si la mejor opción para resolver dicha necesidad es efectivamente un tablero de Power BI. Posterior a ello, le daría prioridad a los BI que le ayudan al proceso en el día a día, cuyo uso es masivo y que pueden llegar a interferir con algún proceso si fallan; luego revisaría los BI que están enfocados en un público gerencial, pues allí se toman decisiones estratégicas para el negocio, y un negocio no puede tomar decisiones sin datos. Finalmente, descartaría migrar los BI que no cumplen con un buen desarrollo o que sencillamente otro tipo de herramienta puede solventar de una manera más eficiente. El orden de prioridad bajo estos criterios sería el siguiente:

ID	Decisión	Prioridad	Por qué
R07 Detalle de casos abiertos	Migrar	1	Es el que más gente usa (25, a diario). Revisaría por qué su último uso fue hace una semana; sin embargo, por la temática es un BI crucial: nos permite saber la cantidad de casos abiertos y entender por qué siguen abiertos.
R06 Indicadores ejecutivos	Migrar	2	Lo ven los ejecutivos a diario, pero depende de una vista vieja que primero hay que entender. Sin embargo, si es un BI para la parte ejecutiva, debe ser prioridad corregirlo y migrarlo.
R05 Clientes sin actividad de prevención	Migrar	3	20 usuarios cada semana y es sencillo.
R01 + R02 Casos por cliente y período	Unir en uno	4	Parecen el mismo reporte; se debe migrar por el nivel de detalle del tema analizado, pero primero hay que analizar cómo unificarlos.
R09 Facturación vs presupuesto	Migrar	5	Es útil: nos permite entender cómo va el negocio versus sus proyecciones y nos indica si vamos o no por buen camino.
R03 Costo acumulado del año	No migrar	Sujeto a revisión	Solo tiene 3 usuarios; se debe revisar si se puede unir a otro visualizador o si se puede usar una herramienta más sencilla.
R08 Análisis histórico 5 años	No migrar	Sujeto a revisión	Posiblemente es un visualizador sin uso; revisaría el historial para llevarlo al Lakehouse, pero no me concentraría en migrarlo.
R04 Costo acumulado año (copia)	No migrar	No aplica	Es una copia de R03 y nadie lo abre desde hace 8 meses.
R10 Prueba indicadores nuevos	No migrar	No aplica	Era una prueba y nadie lo usa desde hace un año. Guardaría las ideas que sirvan.

5.1.2 R01 y R02 parecen ser el mismo reporte. ¿Qué haría antes de migrar cualquiera de los dos?

Compararía cómo calcula cada uno, los correría con los mismos filtros y explicaría cada diferencia. Luego les preguntaría a sus usuarios por qué existe la versión 2. Con eso se acuerda una sola versión con el dueño, se migra esa y se avisa que la otra se retira.

5.1.3 R06 consume una vista legada y R08 viene de Excel. ¿Qué riesgos anticipa en cada caso y cómo los manejaría?

	Riesgo	Qué haría
R06 · vista vieja	La lógica está escondida en la vista y nadie la documentó. Si las cifras cambian, los ejecutivos lo notarán enseguida.	Sacar y documentar lo que hace la vista, rehacerlo en el Lakehouse y correr los dos en paralelo hasta que den igual.
R08 · Excel	Datos puestos a mano, sin revisar, con varias versiones y que dependen de una sola persona.	Buscar de dónde salen y quién es el dueño, cargarlos una vez al Lakehouse revisándolos (como en la Sección 3) y comparar con la base principal.

Sección 6 — Criterio sobre IA aplicada y agentes conversacionales

6.1 Criterio sobre RAG y agentes

La organización quiere que los coordinadores de cuenta puedan preguntarle a un asistente en Teams cosas como: ¿cuántos casos tuvo el cliente X este mes? ¿Cuál es su nivel de riesgo?, sin necesitar abrir Power BI ni llamar al equipo de datos.

6.1.1 ¿Qué es una arquitectura RAG y cómo aplicaría ese concepto para resolver este caso? Explíquelo sin tecnicismos.

Una arquitectura RAG es un sistema en el que el agente busca en memoria la información disponible sobre el tema; no alucina ni inventa datos, sino que responde la pregunta con la información disponible. En este caso aplicaría un agente en el que el modelo semántico sea el insumo principal del RAG, priorizando que el uso de memoria sea lo más eficiente posible para que el modelo sea costo-eficiente.

6.1.2 ¿Por qué conectar ese agente al modelo semántico de Power BI sería mejor que conectarlo directamente a las tablas de la base de datos? ¿Qué ventajas tiene para la consistencia de las respuestas?

Es mejor conectarlo al modelo semántico de Power BI porque este ya contiene la lógica de negocio que la empresa definió para interpretar los datos; además, ya hace todo el procesamiento necesario para tener solo los datos necesarios y verificados. En resumen, la base de datos tiene los datos; el modelo semántico tiene el significado de esos datos.

6.1.3 ¿Cuál es el mayor riesgo de un agente conversacional sobre datos de negocio? ¿Cómo lo mitigaría?

Que responda **algo equivocado con total seguridad** (otro cliente, otro mes, un número inventado) y alguien decida con eso. Para evitarlo:

- Los números salen siempre de una consulta al modelo; el asistente nunca los calcula ni se los inventa.
- Cada respuesta dice qué entendió ('Cliente X, septiembre de 2026') y trae el enlace para verificar.
- Si la pregunta no es clara, pregunta; si no sabe, lo dice.

6.2 Experiencia o acercamiento con IA aplicada

Describe brevemente su experiencia o acercamiento con tecnologías de inteligencia artificial aplicada: modelos de lenguaje, agentes, automatización con IA u otras.

Tengo experiencia en modelos de machine learning y visión artificial, y estoy capacitándome continuamente en agentes y despliegues en la nube, específicamente con Azure DevOps.

6.2.1 ¿Qué herramientas o tecnologías de IA ha usado o explorado? ¿En qué contexto?

- He usado Coco como IA en la creación de vistas en data lakes (revisé y desplegué vistas de datos para mí y para otras personas de la compañía).
- He utilizado Claude Code para realizar prototipos y trabajar conjuntamente con mi equipo (realicé validación de código en distintos proyectos, como páginas web, modelos de visión artificial para la detección de objetos y personas, y modelos de estadística y machine learning).
- He utilizado el asistente de n8n (validación de flujos para agentes conversacionales y conexiones a través de MCP).

6.2.2 Si tuviera que construir un primer prototipo de agente conversacional sobre los datos de esta prueba, ¿por dónde empezaría? ¿Qué herramienta elegiría y por qué?

De hecho, ya realicé el agente sobre estos datos. Primero utilicé el modelo semántico que reposa en la página web; así me aseguré de que la información es la que está validada, en lugar de conectarme a las fuentes de datos sin procesar. Con n8n realicé una conexión a los datos a través de una API key y las credenciales; luego elegí un modelo básico de OpenAI que me permitiera responder preguntas básicas sobre los datos y, finalmente, agregué la memoria para que tuviera el contexto de los chats y lo desplegué gratuitamente.

6.2.3 ¿En qué situaciones NO usaría un agente conversacional y preferiría otro tipo de solución?

- Si siempre se necesita el mismo reporte: un tablero o un correo programado son más baratos y más confiables.
- Si la cifra tiene que ser auditable u oficial.
- Si los datos de base no son confiables: el agente solo entregaría más rápido un número malo.
- Para decisiones importantes sin que una persona revise.